2. Le code

2.1 Les fonctions

Dans le programme que vous rédigerez, vous devrez mettre en place les fonctions suivantes :

1. Lecture des données issues du fichier texte (.txt) et son stockage en mémoire.
2. Affichage des tableaux suivants :
   - ⋆ matrice des coûts
   - ⋆ proposition de transport
   - ⋆ table des coûts potentiels
   - ⋆ table des coûts marginaux

   Attention : la fonction affichage doit être absolument soignée. Toute table comportant des colonnes qui se décalent sera très lourdement sanctionné. La lisibilité des tables est fondamentale.
3. Algorithme pour fixer la proposition initiale : Nord-Ouest.
4. Algorithme pour fixer la proposition initiale : Balas-Hammer.
   - ⋆ Calcul des pénalités.
   - ⋆ Affichage de la (ou des) ligne(s) (ou colonnes) de pénalité maximale.
   - ⋆ Choix de l'arête à remplir.
5. Calcul du coût total pour une proposition de transport donnée.
6. Algorithme de résolution : la méthode du marche-pied avec potentiel.
   - ⋆ Test pour savoir si la proposition est acyclique : on utilisera un parcours en largeur. Lors de

ce parcours, en découvrant les sommets, on vérifiera si on retourne sur un sommet déjà visité et que ce sommet n'est pas le parent du sommet courant, si c'est le cas, alors il existe un cycle. Puis, on affiche le cycle.

- ⋆ Maximisation du transport si un cycle a été détecté. On affiche les conditions pour chacune des cases. Puis on affiche l'arête supprimée (éventuellement plusieurs) à l'issue de la maximisation.
- ⋆ Test pour savoir si la proposition est connexe : on utilisera un parcours en largeur. Si elle est non connexe : affichage des sous graphes connexes composant la proposition.
- ⋆ Modification du graphe s'il est non connexe, jusqu'à l'obtention d'une proposition non dégénérée.
- ⋆ Calcul et affichage des potentiels par sommet.
- ⋆ Affichage des deux tables des coûts potentiels et des marginaux. Détection éventuelle de la meilleure arête améliorante.
- ⋆ Ajout de cette arête améliorante à la proposition de transport, si elle a été détectée.

Attention : Chaque fonction devra être mise en évidence et expliquée clairement à l'oral à l'aide d'un pseudo-code.

Vous devrez impérativement avoir les fonctions suivantes qui marchent :

- ⋆ Lecture des données et son stockage en mémoire.
- ⋆ Algorithmes Nord-Ouest et Balas-Hammer.
- ⋆ Calcul du coût total pour une proposition de transport donnée.
- ⋆ Test pour savoir si la proposition est acyclique en utilisant un parcours en largeur.
- ⋆ Maximisation du transport sur un cycle détecté.
- ⋆ Test pour savoir si la proposition est connexe en utilisant un parcours en largeur.
- ⋆ Affichage de toutes les tables.

2.2 La structure globale

La structure globale de votre programme est illustrée par le pseudo-code suivant :

Début
- Tant que l'utilisateur décide de tester un *problème de transport*, faire :
  - Choisir le numéro du problème à traiter
  - Lire le tableau de contraintes sur fichier et le stocker en mémoire
  - Créer les matrices correspondantes représentant ce tableau et l'afficher
  - Demander à l'utilisateur de choisir l'algorithme pour fixer la proposition initiale et l'exécuter.
  - Afficher les éléments précédemment évoqués lors de l'exécution des deux algorithmes.
  - Dérouler la méthode du marche-pied avec potentiel en affichant à chaque itération :
    - ⋆ Affichage de la proposition de transport, ainsi que son coût de transport total.
    - ⋆ Test pour savoir si la proposition de transport est dégénérée.
    - ⋆ Modifications du graphe de transport pour obtenir un arbre, dans les cas cyclique ou non connexe.
    - ⋆ Calcul et affichage des potentiels.
    - ⋆ Affichage des tables : coûts potentiels et coûts marginaux.
      - ⋆ Si elle n'est pas optimale :
        Affichage de l'arête à ajouter.
        Maximisation du transport sur le cycle formé et une nouvelle itération.
      - ⋆ Sinon sortir de la boucle
      - ⋆ Fin si
  - Afficher la proposition de transport optimale, ainsi que son coût.
  - Proposer à l'utilisateur de changer de problème de transport
- fin Tant que

Fin

2.3 Améliorations possibles

Une fois l'algorithme suffisamment testé, nous vous proposons les améliorations suivantes :

1. Lors du "Modifications du graphe de transport pour obtenir un arbre, dans les cas cyclique ou non connexe", il faudra d'abord détecter si le graphe présente un cycle. Après avoir maximisé sur ce cycle la proposition de transport, il se peut qu'il reste d'autres cycles. Il faudra alors relancer, de manière répétée, les fonctions de "Détection de cycle", puis "Maximisation sur le cycle" jusqu'à l'obtention d'une proposition acyclique. Ce n'est qu'après que l'on effectuera le test de connexité où l'on complétera, au besoin, le graphe avec des arêtes classées selon des coûts croissants jusqu'à l'obtention d'une proposition connexe et acyclique.
2. Lors de l'exécution de la fonction "Maximiser le transport sur un cycle", il se peut que $\delta = 0$, c'est à dire que cela n'induit pas de modification sur le cycle. Vous pouvez alors détecter ce cas particulier. On agira ainsi : on conservera l'arête améliorante détectée avec la table des

coûts marginaux (s'il s'agit de la boucle générale) et on enlèvera l'entièreté des dernières arêtes ajoutées lors du dernier test de connexité, à la même itération. La fonction "Modification du graphe s'il est non connexe" qui suivra proposera alors un ensemble différent d'arêtes.

2.4 Les traces d'exécution

Les traces d'exécution pour les 12 graphes fournis en annexes sont demandés dans le rendu. On appelle trace d'exécution ce qui est affiché par la console. Elles ne pourront pas être remplacées par des copies d'écran.

Il faudra exécuter avec votre programme les 12 problèmes dans les deux cas : proposition initiale Nord-Ouest NO puis Balas-Hammer BH. Puis on affichera le déroulé du marche-pied avec potentiel avec l'entièreté des tableaux et des informations précédemment demandées.

Les fichiers seront stockés de la façon suivante :

- ⋆ Groupe 2 - Equipe 4 - Problème 5 - Nord-Ouest : "2-4-trace5-no.txt"
- ⋆ Groupe 3 - Equipe 2 - Problème 12 - Balas-Hammer : "3-2-trace12-bh.txt"

3. L'étude de la complexité

3.1 Introduction

Cette partie doit être abordée par toutes les équipe, à partir du moment où des fonctions marchent. Nous vous proposons maintenant d'étudier la *complexité* des algorithmes de ce projet.

Mais d'abord qu'est-ce que la complexité d'un algorithme ? Il s'agit de l'évaluation des ressources nécessaires à exécution d'un algorithme (essentiellement la quantité de mémoire requise) et le temps de calcul à prévoir. Ces deux notions dépendent de nombreux paramètres matériels qui sortent du domaine de l'algorithmique : nous ne pouvons attribuer une valeur absolue ni à la quantité de mémoire requise ni au temps d'exécution d'un algorithme donné. En revanche, il est souvent possible d'évaluer l'*ordre de grandeur* de ces deux quantités de manière à identifier l'algorithme le plus efficace au sein d'un ensemble d'algorithmes résolvant le même problème.

C'est ce que nous nous proposons de faire ici en comparant les propositions de transports issues des algorithmes de Nord-Ouest, Balas-Hammer lors de la résolution de la méthode du marche-pied avec potentiel.

3.2 Un peu de théorie

La plupart des algorithmes ont un temps d'exécution qui dépend non seulement de la taille des données en entrée mais des données elles-mêmes. Dans ce cas on distingue plusieurs types de complexités :

Définition 3.1 (complexité dans le pire des cas)
La **complexité dans le pire des cas** est un majorant du temps d'exécution possible pour toutes les entrées possibles d'une même taille. On l'exprime en général à l'aide de la notation O.

Définition 3.2 (complexité dans le meilleur des cas)
La **complexité dans le meilleur des cas** est un minorant du temps d'exécution possible pour toutes les entrées possibles d'une même taille. On l'exprime en général à l'aide de la notation Ω. Cependant cette notion n'est que rarement utilisée car souvent peu pertinente au regard des complexités dans le pire des cas et en moyenne.

Définition 3.3 (complexité en moyenne)
La **complexité en moyenne** est une évaluation du temps d'exécution moyen portant sur toutes les entrées possible d'une même taille supposées équiprobable.

Définition 3.4 (complexité spatiale)
La **complexité spatiale** évalue la consommation en espace mémoire. Le principe est le même sauf qu'ici on cherche à évaluer l'ordre de grandeur du volume en mémoire utilisé : il ne s'agit pas d'évaluer précisément combien d'octets sont consommés par un algorithme mais de préciser son taux de croissance en fonction de la taille n de l'entrée.

3.3 L'étude

Dans ce projet, on analysera la **complexité dans le pire des cas**. Pour cela, nous vous demandons de générer des problèmes de transport aléatoires. Puis de regarder les temps d'exécution des algorithmes.

3.3.1 Les problèmes de transport en entrée

Afin de simplifier le problème, vous travaillerez avec des problèmes de transport de taille $n = m$. La matrice $A = (a_{i,j})_{(i,j)\in[\![1;n]\!]^2}$ est alors carrée.

Afin de générer un échantillonnage mimant toutes les entrées possibles d'une même taille n, vous écrirez une fonction pour éditer des problèmes de transport aléatoires. Elle se fera de la manière suivante :

1. On génère un nombre aléatoire entier entre 1 et 100 inclus pour chaque $a_{i,j}$.
2. On génère un nombre aléatoire entier entre 1 et 100 inclus pour chaque $(temp_{i,j})_{i,j}$ d'une matrice de taille $n \times n$. Vous remplirez ainsi les n valeurs des provisions $(P_i)_{i\in[\![1;n]\!]}$ et des commandes $(C_j)_{j\in[\![1;n]\!]}$ de la façons suivante :

$$P_i = \sum_{j=1}^{n} temp_{i,j} \text{ et } C_j = \sum_{i=1}^{n} temp_{i,j}$$

3.3.2 La mesure du temps

Une fois le problème généré, c'est à dire une fois que les P_i, les C_j et les $a_{i,j}$ sont fixés, il faudra stocker la valeur du temps de chaque portion de code qui nous intéresse. Par exemple en Python, il suffit simplement d'utiliser la fonction time.clock() qui renvoie le temps CPU en secondes et de stocker cette valeur. La différence entre 2 valeurs relevées donnera le temps d'exécution de la portion de code encadrée.

Avec ce problème de taille n généré, vous devrez mesurer le temps d'exécution de :

1. l'algorithme Nord-Ouest. On appellera ce temps $\theta_{NO}(n)$,
2. l'algorithme Balas-Hammer. On appellera ce temps $\theta_{BH}(n)$,
3. l'algorithme du marche-pied avec la proposition issue de Nord-Ouest. On appellera ce temps $t_{NO}(n)$,
4. l'algorithme du marche-pied avec la proposition issue de Balas-Hammer. On appellera ce temps $t_{BH}(n)$,

3.3.3 Le nuage de points

Pour chaque valeurs de n, vous ferez tourner 100 fois votre programme avec des valeurs aléatoires différentes pour le problème de transport. On obtiendra donc, pour n fixé, 100 valeurs de $\theta_{NO}(n)$ par exemple.

Valeurs de n à tester	10	40	10^2	4.10^2	10^3	4.10^3	10^4

Attention : pour effectuer cette tâche, vous travaillerez sur une seule machine à processeur unique. Les instructions seront exécutées l'une après l'autre, sans opération simultanées. Il ne faudra donc pas utiliser votre machine pour faire autre chose pendant toute l'exécution.

Une fois les valeurs stockées, vous tracerez en fonction de n, les nuages de points (les 100 valeurs pour une même abscisse) suivants :

- ⋆ $\theta_{NO}(n)$.
- ⋆ $\theta_{BH}(n)$.
- ⋆ $t_{NO}(n)$.
- ⋆ $t_{BH}(n)$.
- ⋆ $(\theta_{NO}+t_{NO})(n)$.
- ⋆ $(\theta_{BH}+t_{BH})(n)$.

3.3.4 La complexité dans le pire des cas par algorithme

On suppose que la complexité dans le pire des cas est l'enveloppe supérieure du nuage de points. Pour chaque valeur de n, déterminer cette valeur maximale au travers des 100 réalisations pour n fixé. Puis tracer en fonction de n la valeur maximale trouvée.

Pour θ_{NO}, θ_{BH}, t_{NO}, t_{BH}, $(\theta_{NO}+t_{NO})$ et $(\theta_{BH}+t_{BH})$ identifier le type de complexité dans le pire des cas à l'aide du tableau 3.1 :

O($log(n)$)	logarithmique
O(n)	linéaire
O($nlog(n)$)	quasi-linéaire
O(n^2)	quadratique
O(n^k) (k > 2)	polynomiale
O(k^n) (k > 1)	exponentielle

FIGURE 3.1 – Qualifications usuelles des complexités.

3.3.5 Comparaison de la complexité dans le pire des cas

Comparons maintenant les deux algorithmes résolvant le même problème pour n fixé en traçant :

$$\frac{t_{NO}+\theta_{NO}}{t_{BH}+\theta_{BH}}(n)$$

Tracez ensuite la valeur maximale trouvée pour chaque valeur de n et discutez des résultats.